# Structures de Données et Algorithmes Avancés

# TP-1 — Setup de l'Environnement

## Docker, Java, Gradle, JUnit, Python, Git

Master 1 — 2025–2026

Dr. El Hadji Bassirou TOURÉ
Département Génie Informatique
Université Cheikh Anta Diop de Dakar

**Prérequis et contexte**

**Prérequis :** Un ordinateur (Windows, macOS ou Linux) avec au moins 8 Go de RAM et 10 Go d'espace disque libre.

**Objectif :** Installer et vérifier l'environnement de développement standardisé du cours. Ce TP doit être complété **avant** la séance S1.

**Principe :** Vous éditez vos fichiers sur votre machine avec l'éditeur de votre choix. Vous compilez et exécutez dans le conteneur Docker via le terminal.

**Objectifs concrets**

À la fin de ce TP, vous serez capable de :

1. Lancer le conteneur Docker du cours et y accéder
2. Compiler et exécuter un programme Java dans le conteneur
3. Créer un projet Gradle et exécuter `gradle -no-daemon build`
4. Écrire et exécuter un test JUnit 5
5. Générer une figure avec Python/matplotlib
6. Utiliser les commandes Git de base

## Table des matières

**1 Pourquoi Docker ? 3**

**2 Installation de Docker 3**
2.1 Windows . . . 3
2.2 macOS . . . 3
2.3 Linux (Ubuntu/Debian) . . . 3

**3 Premier lancement du conteneur 4**
3.1 Créer un dossier de travail . . . 4
3.2 Lancer le conteneur . . . 4
3.3 Vérifier l'environnement . . . 4

**4 Hello World en Java** **4**

**5 Premier projet Gradle** **5**
5.1 Créer un projet . . . . . 5
5.2 Ajouter une classe . . . . . 5
5.3 Compiler et exécuter . . . . . 5

**6 Premier test JUnit** **6**
6.1 Écrire le test . . . . . 6
6.2 Exécuter les tests . . . . . 6

**7 Vérification Python et matplotlib** **7**

**8 Git — les bases** **8**

**9 Aide-mémoire** **8**

# 1 Pourquoi Docker ?

Tous les étudiants travaillent dans le même environnement, quelle que soit leur machine. Docker crée un **conteneur** — un environnement Linux isolé qui contient tous les outils nécessaires.

| Outil | Usage |
|---|---|
| OpenJDK 17 (Temurin) | Compilation et exécution Java |
| Gradle 8.12 | Construction de projets, dépendances |
| JUnit 5 | Tests unitaires |
| Python 3 | Génération de figures (matplotlib) |
| Git | Contrôle de version |

Le principe est simple : votre dossier de travail est **partagé** entre votre machine et le conteneur. Vous éditez sur votre machine (avec l'éditeur de votre choix : VS Code, Sublime Text, Notepad++...), et vous compilez/testez dans le conteneur.

# 2 Installation de Docker

## 2.1 Windows

1. Télécharger **Docker Desktop** [1]
2. Lancer l'installateur (activer WSL 2 si demandé), redémarrer
3. Installer **Git for Windows** depuis https://gitforwindows.org/
4. Ouvrir **Git Bash** et vérifier :

```
docker --version
```

**Important — Windows**

Sur Windows, utilisez toujours **Git Bash** pour les commandes Docker de ce cours (pas CMD, pas PowerShell).

## 2.2 macOS

1. Télécharger **Docker Desktop**[1]
2. Choisir Apple Silicon ou Intel, installer, lancer
3. Ouvrir Terminal et vérifier : `docker -version`

## 2.3 Linux (Ubuntu/Debian)

```
sudo apt-get update
sudo apt-get install docker.io
sudo usermod -aG docker $USER
# Se déconnecter et se reconnecter, puis vérifier :
docker --version
```

1. https://www.docker.com/products/docker-desktop/

**Vérification 1**

La commande `docker -version` affiche un numéro de version.

# 3 Premier lancement du conteneur

## 3.1 Créer un dossier de travail

Créez un dossier sur votre machine pour tout le cours :

```
mkdir ~/dsa-m1
cd ~/dsa-m1
```

## 3.2 Lancer le conteneur

```
# Linux / macOS
docker run -it -v "$(pwd):/workspace" elbachirtoure/dsa-java-m1:v1.1

# Windows (Git Bash)
MSYS_NO_PATHCONV=1 docker run -it -v "$(pwd):/workspace" \
    elbachirtoure/dsa-java-m1:v1.1
```

La première fois, Docker télécharge l'image (∼800 Mo). Les fois suivantes, le lancement est instantané.

L'option `-v` monte votre dossier local dans `/workspace`. Tout fichier créé d'un côté est visible de l'autre.

**Message d'accueil**

Vous voyez le message d'accueil du cours. Tapez `help-tp` pour l'aide-mémoire.

## 3.3 Vérifier l'environnement

```
cd /workspace
java --version
gradle --version
python3 --version
git --version
```

**Vérification 2**

Java 17, Gradle 8.12, Python 3 et Git sont installés.

# 4 Hello World en Java

Sur votre machine, ouvrez votre éditeur préféré et créez le fichier `Hello.java` dans le dossier `dsa-m1/` :

```
1 public class Hello {
2     public static void main(String[] args) {
3         System.out.println("Structures de Donnees - M1");
4         System.out.println("Environnement pret !");
```

```
    }
}
```

Dans le terminal du conteneur :

```
cd /workspace
javac Hello.java
java Hello
```

**Vérification 3**

Le programme affiche les deux lignes. Modifiez-le pour afficher votre nom, recompilez et exécutez.

# 5 Premier projet Gradle

Gradle gère la compilation, les dépendances (JUnit) et les tests.

## 5.1 Créer un projet

```
cd /workspace
new-project hello-dsa
cd hello-dsa
```

Structure créée :

```
hello-dsa/
  src/main/java/        # Code source
  src/test/java/        # Tests JUnit
  build.gradle           # Configuration
```

## 5.2 Ajouter une classe

Sur votre machine, créez `hello-dsa/src/main/java/Main.java` :

```
public class Main {
    private String nom;

    public Main(String nom) {
        this.nom = nom;
    }

    public String message() {
        return "Bienvenue en DSA, " + nom + " !";
    }

    public static void main(String[] args) {
        Main s = new Main("Fatou");
        System.out.println(s.message());
    }
}
```

## 5.3 Compiler et exécuter

```
gradle --no-daemon build
gradle --no-daemon run
```

**Pourquoi `--no-daemon` ?**

Sur les volumes Docker partagés avec Windows, le daemon Gradle peut provoquer des erreurs d'entrée/sortie. L'option `--no-daemon` évite ce problème. Le premier build prend 10–20 secondes, les suivants seront plus rapides.

**Vérification 4**

`gradle -no-daemon run` affiche `Bienvenue en DSA, Fatou !`

# 6 Premier test JUnit

Un test vérifie qu'une méthode fait ce qu'elle est censée faire.

## 6.1 Écrire le test

Créez `hello-dsa/src/test/java/MainTest.java` :

```java
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class MainTest {

    @Test
    void testMessage() {
        Main s = new Main("Moussa");
        assertEquals("Bienvenue en DSA, Moussa !", s.message());
    }

    @Test
    void testAutreNom() {
        Main s = new Main("Aminata");
        assertTrue(s.message().contains("Aminata"));
    }

    @Test
    void testNonVide() {
        Main s = new Main("Cheikh");
        assertFalse(s.message().isEmpty());
    }
}
```

## 6.2 Exécuter les tests

```
gradle --no-daemon test
```

Résultat attendu :

```
MainTest > testMessage() PASSED
MainTest > testAutreNom() PASSED
```

```
MainTest > testNonVide() PASSED

BUILD SUCCESSFUL
```

**JUnit — les assertions essentielles**

- `assertEquals(attendu, obtenu)` — vérifie l'égalité
- `assertTrue(condition)` — vérifie qu'une condition est vraie
- `assertFalse(condition)` — vérifie qu'une condition est fausse
- `assertThrows(Exception.class, () -> ...)` — vérifie qu'une exception est levée

**Vérification 5**

Les 3 tests passent. Ajoutez un 4e test de votre choix et relancez `gradle -no-daemon test`.

## 7 Vérification Python et matplotlib

Python génère les figures du cours. Créez `test_figure.py` dans `dsa-m1/` :

```python
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import numpy as np

n = np.arange(1, 50)
fig, ax = plt.subplots(figsize=(8, 5))
ax.plot(n, np.ones_like(n), label='O(1)', linewidth=2)
ax.plot(n, np.log2(n), label='O(log n)', linewidth=2)
ax.plot(n, n, label='O(n)', linewidth=2)
ax.plot(n, n * np.log2(n), label='O(n log n)', linewidth=2)

ax.set_xlabel('n')
ax.set_ylabel("Operations")
ax.set_title('Comparaison des complexites')
ax.legend()
ax.set_ylim(0, 200)
ax.grid(True, alpha=0.3)
plt.tight_layout()
fig.savefig('test_complexites.png', dpi=150)
print("Figure sauvegardee : test_complexites.png")
```

```
cd /workspace
python3 test_figure.py
```

Le fichier `test_complexites.png` apparaît dans votre dossier `dsa-m1/` — ouvrez-le sur votre machine.

**Vérification 6**

La figure affiche 4 courbes de complexité correctement étiquetées.

# 8 Git — les bases

```
git config --global user.name "Votre Nom"
git config --global user.email "votre.email@ucad.edu.sn"

cd /workspace/hello-dsa
git init
git add .
git commit -m "TP-1 : premier projet avec JUnit"
git log --oneline
```

**Branche par défaut**

Le conteneur est configuré pour que `git init` crée une branche `main` (et non `master`).

Modifiez un fichier, puis :

```
git add .
git commit -m "TP-1 : ajout d'un test"
git log --oneline
```

**Vérification 7**

`git log -oneline` affiche 2 commits.

# 9 Aide-mémoire

**Commandes Docker**

| Commande | Description |
|---|---|
| `docker run -it -v "$(pwd):/workspace" elbachirtoure/dsa-java-m1:v1.1` | Lancer le conteneur |
| `exit` | Quitter le conteneur |
| `help-tp` | Aide-mémoire (dans le conteneur) |
| `new-project <nom>` | Créer un projet Gradle |

**Outils dans le conteneur**

| Outil | Commande | Usage |
|---|---|---|
| Java | `javac Fichier.java` | Compiler |
| | `java Fichier` | Exécuter |
| Gradle | `gradle -no-daemon build` | Compiler + tests |
| | `gradle -no-daemon test` | Tests uniquement |
| | `gradle -no-daemon run` | Exécuter le main |
| Python | `python3 script.py` | Exécuter un script |
| Git | `git add . && git commit -m "msg"` | Committer |

## Rapport individuel

**Rapport individuel — obligatoire**

Chaque étudiant rédige un rapport de **1 à 2 pages** :

1. **Ce que j'ai fait :** outils installés, programmes écrits
2. **Les difficultés :** erreurs rencontrées
3. **Comment je les ai résolues**
4. **Captures d'écran :**
   (a) Message d'accueil du conteneur
   (b) Résultat de `gradle -no-daemon test` avec les tests passés
   (c) La figure `test_complexites.png`

**Format :** PDF. **Nom :** `rapport-tp1-PRENOM-NOM.pdf`
**Remise :** Avant la séance S1.